

CHANDIGARH
UNIVERSITY
Discover. Learn. Empower.

CHANDIGARH UNIVERSITY
Discover. Learn. Empower.

EXPERIMENT – 3

NAME: Kanishk Jamwal	**UID: 24BAI70827**
BRANCH: CSE-AIML (CS221)	**SECTION/GROUP:** 24AIT_NTPP3[A]
SEMESTER: 5TH	**DATE OF PERFORMANCE:** 22/07/2026
SUBJECT: Full Stack - II	**SUBJECT CODE:** 24CSP-337

EXPERIMENT 3.1 1.

AIM

To design and implement a secure authentication system using JWT for user login and session management.

2. OBJECTIVE

1. To understand authentication mechanisms in web applications
2. To implement token-based authentication using JWT
3. To manage user sessions in a stateless architecture 4. To handle token storage and validation securely

3. SOFTWARE REQUIREMENT

- React.js
- Node.js (optional/mock-based initially)
- JWT libraries (conceptual understanding)
- Browser (for testing)

4. THEORY

Authentication is the process of verifying the identity of a user before granting access to system resources. Traditional session-based authentication relies on server-side session storage, which can become inefficient in distributed systems.

JWT-based authentication introduces a **stateless approach**, where the server issues a signed token containing user information. This token is stored on the client and sent with each request for verification. A JWT consists of three parts:

- Header – Defines the algorithm
- Payload – Contains user data (claims)
- Signature – Ensures integrity and authenticity

5. IMPLEMENTATION/PROCEDURE

1. Create login form UI
2. Validate user credentials *(mock or static)*
3. Generate or simulate JWT token
4. Store token (localStorage/sessionStorage)
5. Attach token to requests
6. Decode token to extract user information

6. **CODE**

Authentication Service (authService.js) export const

```
authenticateUser = (username, password) => {   const
validUser = {    username: "24bai70882",    password:
"naveen123",    email: "24bai70882@cuchd.in",
```

```
    role: "Administrator",
  };   if
(
   username === validUser.username &&
password === validUser.password
 ) {    const header = btoa(
JSON.stringify({       alg:
"HS256",       typ:
"JWT",
    })
   );

   const            payload =          btoa(
JSON.stringify({       username:
validUser.username,       email: validUser.email,
role:
validUser.role,
    })
   );

   const signature = btoa("jwt-signature");

   return ${header}.${payload}.${signature};
 }

 return null;
};
```

Login Authentication (Login.js) const handleLogin = () => {
const token = authenticateUser(username, password);

```
 if (token) {
localStorage.setItem("token", token);    login();
} else {
   setError("Invalid Username or Password");
 }
};
```

Store JWT Token localStorage.setItem("token",
token);

Decode JWT Token (token.js) export const
getUserFromToken = () => {
const token = localStorage.getItem("token");

```
if (!token) return null;
```

```
  const payload = JSON.parse(
atob(token.split(".")[1])
  );

  return payload;
};
```

Dashboard

```
const user =
getUserFromToken();

<h2>{user?.username}</h2>

<p>{user?.email}</p>

<p>{user?.role}</p>
```

Logout

```
const logout = () => {
  localStorage.removeItem("token");
setIsLoggedIn(false);
};
```

App Component

```
{
isLoggedIn
   ? <Dashboard logout={logout}/>
: <Login login={login}/>
}
```

OUTPUT

The JWT Authentication system was successfully implemented. Users were able to log in using valid credentials, after which a JWT token was generated and securely stored in the browser's localStorage. The application successfully decoded the token to retrieve user information and displayed it on the dashboard. Invalid login attempts were rejected with an appropriate error message, and the logout functionality removed the token, terminating the user session.

7.


RBAC Login
Username
24bai70882
Password
Login
admin / admin123 · editor / editor123 · viewer / viewer123

Fig. 3.1 JWT Authentication system

8. LEARNING OUTCOMES

- Understand the concept of authentication in web applications.
- Implement JWT-based authentication for secure user login.
- Generate and manage JWT tokens for stateless session handling.
- Store authentication tokens securely using localStorage.
- Decode JWT tokens to retrieve user information.
- Implement login, logout, and session management functionality.
- Develop secure authentication workflows in React applications.

1.

2. CHANDIGARH UNIVERSITY
Discover. Learn. Empower.

EXPERIMENT 3.2 AIM

3. To implement role-based access control and secure application routes based on user permissions.

OBJECTIVE

1. To understand authorization mechanisms in applications
2. To implement RBAC for access control

4. 3. To protect routes based on user roles

4. To dynamically render UI elements based on permissions

SOFTWARE REQUIREMENT

1. React.js
2. React Router
3. State management (Context API / Redux)

THEORY

While authentication verifies user identity, **authorization** determines what actions a user is permitted to perform. Role-Based Access Control (RBAC) is a widely used model where users are assigned roles, and each role has defined permissions. Common roles include:

- Admin
- Editor
- Viewer RBAC ensures:

5.

- Controlled access to sensitive features
- Simplified permission management
- Improved system security
  Route protection is implemented to restrict navigation to certain pages unless the user has appropriate permissions.

IMPLEMENTATION/PROCEDURE

6. 1. Define user roles (Admin, Editor, Viewer)
2. Store role in authentication state
3. Implement protected routes using React Router
4. Restrict access based on role
5. Conditionally render UI elements (buttons, pages)
6. Redirect unauthorized users

CODE

User Authentication (authService.js) const users
= [

{

```
    username:       "admin",
password:           "admin123",
role: "Admin",
  },  {    username: "editor",
password:
"editor123",
```

role: "Editor",

```
  },
  {
   username:          "viewer",    password:
    "viewer123",    role: "Viewer",
  },
];

export const authenticateUser = (username, password) => {   const user =
users.find(
   (u) =>
    u.username === username &&
    u.password === password
  );

  if (!user) return null;

  const header = btoa(
JSON.stringify({      alg: "HS256",
typ:
"JWT",
   })
  );

                const    payload    =    btoa(
JSON.stringify({      username:
   user.username,      role: user.role,
   })
  );

  const signature = btoa("rbac-secret");

  return ${header}.${payload}.${signature};
};
```

Login Authentication (Login.js)

```
const
handleLogin = () => {

  const token = authenticateUser(    username,
password
  );

  if (!token) {     setError("Invalid Username or
Password");
return;   }
```

CHANDIGARH UNIVERSITY

Discover. Learn. Empower.

```
  localStorage.setItem("token", token);

  const payload = JSON.parse(     atob(token.split(".")[1])
  );

  if (payload.role === "Admin")      navigate("/admin");

  else if (payload.role === "Editor")      navigate("/editor");

  else      navigate("/viewer");

};
```

Token Utilities (token.js) export const
getUser = () => {

```
  const token = localStorage.getItem("token");

  if (!token) return null;

  return JSON.parse(     atob(token.split(".")[1])
  );
};

export const getRole = () => {

  const user = getUser();

  return user?.role;

};

export const logout = () => {

  localStorage.removeItem("token");

};
```

Protected Route (ProtectedRoute.js) function
ProtectedRoute({ children, allowedRoles,
}) {

```
 if (!isAuthenticated()) {     return <Navigate
to="/" />;
 }

                    const role = getRole();

 if (!allowedRoles.includes(role)) {     return <Navigate
to="/unauthorized" />;
 }

 return children;

}
```

Route Configuration (App.js)

```
                <BrowserRouter>

 <Routes>

  <Route       path="/"       element={<Login />}
  />
  <Route       path="/admin"       element={
<ProtectedRoute
allowedRoles={["Admin"]}
      >
                        <Admin />
                       </ProtectedRoute>
    }
  />
  <Route       path="/editor"       element={
<ProtectedRoute
allowedRoles={["Editor"]}
      >
       <Editor />
                       </ProtectedRoute>
    }
  />
  <Route
```

```
     path="/viewer"
element={
      <ProtectedRoute
allowedRoles={["Viewer"]}
      >
        <Viewer />
      </ProtectedRoute>
    }
  />
  <Route      path="/unauthorized"
element={<Unauthorized />}
  />
 </Routes> </BrowserRouter>
```

Role-Based Dashboards

Admin Dashboard

- User Management
- Reports
- Settings
- Logout

Editor Dashboard

- Manage Content
- Edit Articles
- View Reports
- Logout

Viewer Dashboard

- View Content
- View Reports
- Profile
- Logout

Navigation Bar

<Navbar /> User
Name
User Role
Logout Button

OUTPUT

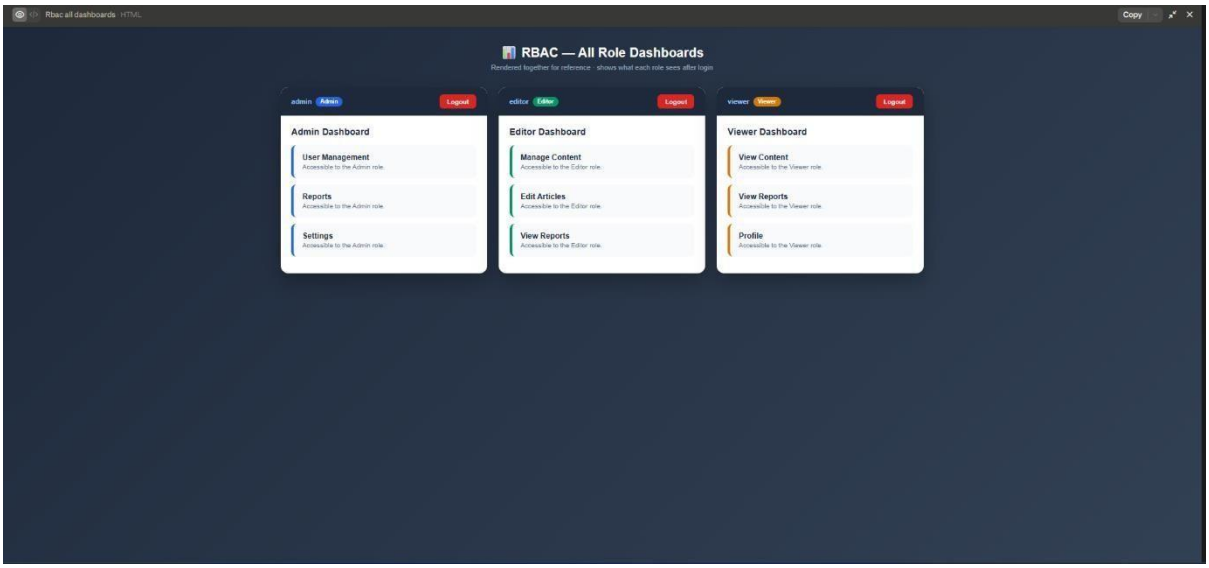
The Role-Based Access Control (RBAC) system was successfully implemented. Users were authenticated using their credentials and redirected to dashboards based on their assigned roles (Admin, Editor, or Viewer). Protected

7. routes restricted unauthorized access, and users attempting to access restricted pages were redirected to the

Unauthorized (403 Access Denied) page. The application dynamically displayed different interfaces according to user permissions, demonstrating secure authorization and role-based navigation.

CHANDIGARH UNIVERSITY

Discover. Learn. Empower.



8. LEARNING OUTCOMES

- Understand the concept of authorization and Role-Based Access Control (RBAC).
- Implement protected routes using React Router.
- Restrict application access based on user roles.
- Dynamically render user interfaces according to permissions.
- Manage user roles and secure navigation in React applications.
- Implement role-based authentication and authorization for enhanced application security.
- Develop secure, scalable, and permission-based web applications.



CHANDIGARH
UNIVERSITY

Discover. Learn. Empower.